VALEMONT OS · MISE EN SERVICE

Le plan et le tuto de toutes les connexions

English version: docs/SETUP_GUIDE.en.md

Pour Théo et Pablo. Ce guide te dit **dans quel ordre** brancher chaque service, **où cliquer**, **quoi coller** et **comment vérifier** que ça marche. Il est écrit à partir des documents du projet (ils font foi) : `.env.example`, `docs/DEPLOYMENT.md`, `docs/INTEGRATIONS.md`, `docs/INBOX.md`, `docs/TRACKER.md`, `docs/SECURITY.md`, `docs/ACCEPTANCE.md`, `docs/DECISIONS.md` (D086, D087). Les étapes chez Google, Vercel, Wise, Stripe, Meta et LinkedIn ont été vérifiées dans leurs documentations officielles en novembre 2026 (liens dans chaque section). Ces interfaces changent souvent : si un menu a un autre nom, cherche le mot-clé indiqué.

Trois règles avant de commencer

1. **Une clé, un jeton ou un mot de passe ne se colle qu'à deux endroits possibles** : dans l'application (**Réglages › Intégrations › Configurer**, où il est chiffré), ou, pour les quelques réglages de base, dans **Vercel › Settings › Environment Variables**. Jamais dans un chat (y compris avec Claude), jamais dans un e-mail, jamais directement dans le Google Sheet, jamais dans le code.
2. **Tous les comptes appartiennent à Valemont** (vos adresses Google, votre Business Manager Meta, votre compte Vercel). Le compte Claude qui a construit l'application n'est jamais utilisé sur vos vraies ressources.
3. **Ce que tu enregistres dans l'application marche tout de suite** : pas de redéploiement. Seul un changement dans Vercel demande **Vercel › Deployments › ⋯ › Redeploy**.

Dans l'application, tout se contrôle dans **Réglages › Intégrations** : une ligne par service (16 au total), avec son statut (*Connecté*, *Non configuré*, *Erreur*, *À renouveler*, *Import uniquement*, *En attente d'approbation*, *Démo, simulé*), les **noms** des valeurs (« défini dans Vercel », « enregistré dans l'app » ou manquant, jamais la valeur elle-même) et les boutons **Configurer**, **Tester la connexion**, **Synchroniser**, **Autoriser / Autoriser à nouveau** quand ils ont un sens. En bas, **Réglages de base (définis dans Vercel)** liste en lecture seule ce qui reste dans Vercel.

---

1. Le plan

Coche au fur et à mesure. « Qui » est une proposition : l'important est que la personne ait les droits d'administration sur le compte concerné.

Brancher un service, c'est toujours la même chose (Phases 4 à 9)

1. **Réglages** › **Intégrations** › ligne du service › **Configurer**.
2. Suis les étapes numérotées (chacune a un lien vers la bonne page du fournisseur ; les adresses à copier chez lui sont affichées avec un bouton Copier).
3. **Colle** ce que le fournisseur te donne, ou clique sur **Générer** pour un secret que l'on crée nous-mêmes : il s'affiche **une seule fois**, copie-le tout de suite dans l'autre service.
4. **Enregistrer et tester** : vert = « Ça marche. », rouge = la raison en clair. Rien à redéployer.

Une variable du même nom dans Vercel reste possible (ancienne méthode) : **une variable dans Vercel a priorité**, et le formulaire l'affiche alors comme « défini dans Vercel » sans pouvoir l'écraser.

Phase 0 · Prérequis et comptes (les deux · 30 min)

- ☐ Vérifier que vous avez : un compte Google chacun (idéalement Google Workspace, même domaine), un compte GitHub, un compte Vercel, l'accès administrateur Wise Business, Stripe, Meta Business (Page Facebook + comptes Instagram professionnels), les Pages LinkedIn, GA4, Search Console, l'hébergement des sites (Netlify ou autre), et Make, Zapier ou n8n si vous voulez le relais des messages.
- ☐ Rien à installer : ni terminal, ni Node.js. Tout se fait dans le navigateur.
- ☐ Un gestionnaire de mots de passe partagé (1Password, Bitwarden…) pour garder une copie des réglages de base.

Débloque : rien encore, mais évite d'être bloqué au milieu.

Phase 1 · Google : projet, Sheet, connexion (Théo, ou le titulaire du compte Google admin · 1 h 30)

- ☐ Créer le projet Google Cloud et activer les 4 API (Sheets, Drive, Calendar, Gmail) (§2.2).
- ☐ Créer le **compte de service** et télécharger sa clé JSON (§2.2).
- ☐ Créer le Google Sheet principal vide et le partager avec le compte de service (§2.3).
- ☐ (Facultatif) Chacun crée son Sheet privé et le partage (§2.3).

- [ ] Partager le dossier Drive « Valemont OS » (Lecteur), préparer le dossier des sauvegardes, partager les agendas (§2.5).
- [ ] Écran de consentement OAuth + client OAuth (§2.4) ; les adresses de redirection exactes viendront en Phase 2.

Débloque : la base de données (Sheet), la connexion Google des deux fondateurs, Fichiers, Agenda.

Phase 2 · Mise en ligne (Théo ou Pablo · 30 min)

- [ ] Cliquer sur le bouton **Deploy with Vercel** du README et remplir le formulaire avec **les réglages de base seulement** : `APP_MODE` = `live`, `SESSION_SECRET`, vos deux adresses Google, l'ID et le secret du client OAuth, l'ID du Sheet, la clé JSON du compte de service (collée telle quelle). Plan **Pro**, région Paris (§2.7).
- [ ] Ouvrir `https://<votre-app>/setup` : la page dit ce qui manque encore (noms seulement, jamais de valeur) et affiche les 3 adresses de redirection à coller dans Google Cloud (§2.4).
- [ ] Sur `/setup`, cliquer sur **Générer un secret** pour créer `CONNECTORS_ENCRYPTION_KEY` (la clé qui chiffre ce que vous collerez dans l'app) et, avec un nouveau clic, `CRON_SECRET` ; les ajouter dans Vercel, en garder une copie dans le gestionnaire de mots de passe, **Redeploy** une fois, puis **Vérifier à nouveau**.

Débloque : l'application en ligne en mode LIVE (sans les réglages de base, elle affiche une page de configuration, jamais la démo). `APP_BASE_URL` se règle tout seul sur Vercel.

Phase 3 · Premier démarrage (les deux · 10 min)

- [ ] Se connecter avec Google : on arrive sur la page **Installation** › **Préparer la base de données**. L'application crée les onglets et en-têtes du Sheet, puis vérifie tout. Ensuite, Ma journée s'ouvre (§2.8).
- [ ] Théo se connecte, Pablo se connecte dans un autre navigateur ; un compte non fondateur est refusé.
- [ ] Réglages › Calendriers et Réglages › Espace de travail : coller les identifiants d'agendas et du dossier Drive.

Débloque : toute l'application de base (Ma journée, CRM, Appels, Tâches, Agenda, Fichiers).

Phase 4 · Argent : Wise et Stripe (celui qui administre la banque · 45 min)

- ☐ Wise › **Configurer** : jeton en lecture seule (+ webhook facultatif) › **Enregistrer et tester** (§2.9).
- ☐ Stripe › **Configurer** : clé restreinte en lecture + secret du webhook › **Enregistrer et tester** (§2.10).

Débloque : Finances (soldes, mouvements, ventes, frais, virements), MRR dans Finances et le Command Center, taux USD→EUR pour les paiements fournisseurs en dollars. L'application ne peut jamais déplacer d'argent.

Phase 5 · Mails (Gmail) et agenda (chacun pour soi · 10 min chacun, + délai Google éventuel)

- ☐ Vérifier que `gmail.send` est dans l'écran de consentement, puis chacun clique **Autoriser** sur la ligne Gmail.
- ☐ Vérifier que l'Agenda fusionne bien les agendas (l'agenda perso de l'autre apparaît en « Occupé »).

Débloque : envoi d'e-mails en un clic depuis Appels et Croissance › Messages (toujours après ton clic). ⏳ Avec une application Google « Externe », le consentement est redemandé souvent et la publication en production demande une **vérification Google** (voir §2.6).

Phase 6 · Site web et messages à traiter (Pablo ou celui qui gère les sites · 1 h)

- ☐ Formulaires du site › **Configurer** › **Générer** › copier le secret chez l'hébergeur du site (§2.11).
- ☐ Messages à traiter › **Configurer** › **Générer** › copier le secret dans Make / Zapier / n8n (§2.12).

Débloque : chaque formulaire de site devient un prospect entrant + un élément « À répondre » ; les commentaires et messages arrivent dans Ma journée.

Phase 7 · Réseaux : Instagram, LinkedIn, GA4, Search Console (Théo ou Pablo · 2 h + délais Meta/LinkedIn)

- ☐ Instagram › **Configurer** : jeton (utilisateur système) + identifiant du compte › **Enregistrer et tester** (§2.13).

- [ ] Instagram : webhook commentaires/messages → Messages à traiter (§2.12 B). ⏳ **Examen de l'app par Meta** pour les comptes qui n'ont pas de rôle dans l'app.
- [ ] LinkedIn : imports d'exports (.xls Page, .xlsx créateur). ⏳ API Community Management : **demande d'accès** facultative, longue et non garantie.
- [ ] GA4 et Search Console : exports CSV importés chaque semaine.

Débloque : Croissance (canaux, coûts, entonnoirs), Studio contenu (chiffres par post), Croissance › SEO.

Phase 8 · Tracker perso (chacun pour soi · 15 min)

- [ ] Mon tracker › **Configurer** : coller l'adresse de ton tracker, **Générer** le secret (et le jeton du flux), les copier dans ton tracker › **Enregistrer et tester** (§2.16).

Débloque : chacune de tes activités envoyée à ton tracker personnel, et un flux qu'il peut lire.

Phase 9 · Sauvegardes et test de restauration (Théo · 30 min)

- [ ] Sauvegardes › **Configurer** : coller l'adresse du dossier Drive des sauvegardes › **Enregistrer et tester** (§2.17).
- [ ] (Facultatif) Capture planifiée › **Configurer**, seulement si un planificateur autre que Vercel l'appelle.
- [ ] Le lendemain : Réglages › Diagnostic › Sauvegardes › **Vérifier** sur la copie du jour (exercice de restauration).

Débloque : copie quotidienne vérifiée du Sheet, capture Instagram toutes les 6 h, plan B en cas de pépin.

Phase 10 · Vérifications finales (les deux · 45 min)

- [ ] Réglages › Intégrations : chaque ligne configurée est **Connecté** après **Tester la connexion** ; GA4 et Search Console = *Import uniquement* (normal) ; LinkedIn = *En attente d'approbation* (normal).
- [ ] Réglages › Diagnostic : santé « prête », sauvegardes « vérifiée ».
- [ ] Dérouler la liste « Stage 4 gate: live validation » de `docs/ACCEPTANCE.md` et noter la date de chaque case dans `docs/STATUS.md` (rien n'est VERIFIED_LIVE avant ça).

2. Le tuto

2.1 Les secrets : qui les crée, où ils vont

Tu n'as plus besoin de terminal pour fabriquer un secret :

Secret	Comment l'obtenir	Où il va
`SESSION_SECRET`	Générateur de ton gestionnaire de mots de passe (48 caractères ou plus), ou bouton **Générer un secret** de la page `/setup`	Vercel (formulaire du bouton Deploy)
`CONNECTORS_ENCRYPTION_KEY`, `CRON_SECRET`	Bouton **Générer un secret** de `/setup` (un clic par variable, une valeur différente à chaque fois ; fabriqué dans ton navigateur, envoyé nulle part)	Vercel + copie dans le gestionnaire de mots de passe
Secrets des services (messages, site, tracker, tâches)	Bouton **Générer** dans **Configurer** : créé sur le serveur, enregistré chiffré, affiché **une seule fois**	Copie-le aussitôt dans l'autre service (relais, hébergeur du site, tracker, planificateur)
Clés données par un fournisseur (Wise, Stripe, Meta)	Chez le fournisseur (étapes dans chaque section)	Colle-les dans **Configurer**

Un secret par usage : ne réutilise jamais le même pour deux variables. Garde `CONNECTORS_ENCRYPTION_KEY` dans le gestionnaire de mots de passe : sans elle, les clés enregistrées dans l'app deviennent illisibles (il faudrait les recoller).

Avancé (facultatif) : en terminal, `openssl rand -base64 48` (`SESSION_SECRET`), `openssl rand -base64 32` (`CONNECTORS_ENCRYPTION_KEY`), `openssl rand -hex 32` (autres secrets) donnent le même résultat.

2.2 Google Cloud : projet, API, compte de service

À quoi ça sert : le compte de service est le « robot » qui lit et écrit le Google Sheet (seule base de données), lit le dossier Drive partagé, lit les agendas et fait les sauvegardes.

Ce qu'il te faut : ton compte Google (de préférence Workspace), 30 minutes.

Étapes

1. Va sur https://console.cloud.google.com › sélecteur de projet › **Nouveau projet** › nom « Valemont OS ». (Doc : https://developers.google.com/workspace/guides/create-project)

2. **API et services › Bibliothèque** : active **Google Sheets API**, **Google Drive API**, **Google Calendar API**, **Gmail API**. (Doc : https://developers.google.com/workspace/guides/enable-apis)
3. **IAM et administration › Comptes de service › Créer un compte de service** : nom `valemont-os`, **aucun rôle** (ses droits viennent uniquement des partages que tu fais). Note son adresse : `valemont-os@<projet>.iam.gserviceaccount.com`. (Doc : https://cloud.google.com/iam/docs/service-accounts-create)
4. Ouvre le compte de service › onglet **Clés › Ajouter une clé › Créer une clé › JSON**. Un fichier se télécharge. (Doc : https://cloud.google.com/iam/docs/keys-create-delete)
5. Ouvre le fichier avec un éditeur de texte (Bloc-notes, TextEdit), copie **tout** son contenu.
6. Colle-le tel quel dans le formulaire du bouton Deploy (Phase 2) sous `GOOGLE_SERVICE_ACCOUNT_KEY_BASE64` (le base64 du fichier marche aussi), puis **supprime le fichier** (et vide la corbeille).

Où ça va : Vercel (réglage de base) : `GOOGLE_SERVICE_ACCOUNT_KEY_BASE64`.

Problèmes fréquents

- *« La création de clés de compte de service est désactivée »* : les organisations Google Cloud récentes bloquent les clés par défaut (règle `iam.disableServiceAccountKeyCreation`). L'administrateur Workspace doit lever cette règle **pour ce projet seulement** (IAM › Règles d'administration), créer la clé, puis la remettre.
- Clé perdue ou fuitée : crée une nouvelle clé, remplace la variable dans Vercel, Redeploy, **puis** supprime l'ancienne clé.

2.3 Google Sheets (base de données) et Sheets privés

À quoi ça sert : tout ce que vous saisissez est lu et écrit ici. Les Sheets privés gardent les tâches privées et candidatures de chacun, invisibles pour l'autre.

Étapes

1. Sur https://sheets.google.com, crée un tableur **vide** « Valemont OS » (l'application ne le crée jamais).
2. **Partager** › colle l'adresse du compte de service › rôle **Éditeur** › décoche « Envoyer une notification ».
3. Copie l'identifiant dans l'URL : `https://docs.google.com/spreadsheets/d/<CET-IDENTIFIANT>/edit` → `SHEETS_SPREADSHEET_ID`.
4. (Facultatif) **Chacun** crée **depuis son propre compte** un tableur « Valemont OS – privé <prénom> », le partage avec le compte de service en **Éditeur** (et avec personne d'autre) → `PRIVATE_SHEET_ID_THEO`, `PRIVATE_SHEET_ID_PABLO` (à ajouter dans Vercel, puis Redeploy).
5. Les onglets et en-têtes sont créés en Phase 3 par le bouton **Préparer la base de données** de l'application.

Où ça va : Vercel (réglages de base) : `SHEETS_SPREADSHEET_ID` (obligatoire), `PRIVATE_SHEET_ID_THEO`, `PRIVATE_SHEET_ID_PABLO` (facultatives).

Vérifier : Réglages › Diagnostic › Vérification de la base de données › **Vérifier** : tout en « OK ».

Problèmes fréquents

- *Permission refusée* : le partage a été fait avec une autre adresse, ou en Lecteur au lieu d'Éditeur.
- Ne renomme pas et ne supprime pas les onglets/colonnes à la main : l'application retrouve les colonnes par leur nom. Ajouter des lignes de mise en forme ou des formules dans le Sheet casse la lecture (Diagnostic le signale).
- L'onglet `IntegrationSecrets` ne contient que du texte chiffré illisible : n'y touche pas (une cellule modifiée ou déplacée ne se déchiffre plus, il faudrait recoller la clé dans **Configurer**).

2.4 Google Sign-In (connexion des deux fondateurs)

À quoi ça sert : seules vos deux adresses Google peuvent se connecter. Le même client OAuth sert aussi aux autorisations Drive (`drive.file`) et Gmail (`gmail.send`), demandées plus tard, une par une.

Étapes (le menu s'appelle maintenant **Google Auth Platform** dans la console)

1. **Google Auth Platform › Branding** : nom « Valemont OS », adresse d'assistance = l'adresse d'un fondateur, domaine autorisé = votre domaine. (Doc : https://developers.google.com/workspace/guides/configure-oauth-consent)
2. **Audience** :
   - si vous êtes **tous les deux dans le même Google Workspace** : type **Interne** (le plus simple : pas de vérification, pas d'utilisateurs de test) ;
   - sinon : type **Externe**, statut **Test**, et ajoute vos deux adresses dans **Utilisateurs de test**.
3. **Accès aux données › Ajouter ou supprimer des champs d'application** : `openid`, `.../auth/userinfo.email`, `.../auth/userinfo.profile`, `.../auth/drive.file`, `.../auth/gmail.send`. (Les deux derniers ne sont demandés qu'au moment où tu t'en sers : c'est l'autorisation incrémentale, https://developers.google.com/identity/protocols/oauth2/web-server#incrementalAuth)
4. **Clients › Créer un client** › type **Application Web** › nom « Valemont OS ». **URI de redirection autorisés** : copie les trois adresses exactes affichées par la page `/setup` de l'application (Phase 2). Elles ont cette forme (sans `/` final) :
   - `https://<votre-app>/api/auth/google/callback`
   - `https://<votre-app>/api/auth/google/drive/callback`
   - `https://<votre-app>/api/auth/google/gmail/callback` Si tu crées le client avant le déploiement, reviens ajouter ces adresses ensuite.
5. Copie l'**ID client** → `GOOGLE_OAUTH_CLIENT_ID` et le **code secret** → `GOOGLE_OAUTH_CLIENT_SECRET` (directement dans le formulaire du bouton Deploy).

Où ça va : Vercel (réglages de base) : `GOOGLE_OAUTH_CLIENT_ID`, `GOOGLE_OAUTH_CLIENT_SECRET`, `FOUNDER_THEO_EMAIL`, `FOUNDER_PABLO_EMAIL` ; facultatives : `FOUNDER_THEO_TIMEZONE` (défaut `Europe/Paris`), `FOUNDER_PABLO_TIMEZONE` (défaut `Europe/Madrid`), `APP_BASE_URL` (automatique sur Vercel).

Vérifier : ouvrir l'application, « Se connecter avec Google » : Théo et Pablo entrent, une troisième adresse est refusée.

Problèmes fréquents

- `redirect_uri_mismatch` : l'URI enregistrée ne correspond pas exactement à l'adresse de l'application (https, pas de `/` final, même sous-domaine). Si tu changes de domaine, ajoute les 3 nouvelles URI (la page `/setup` les donne).
- *« Accès bloqué : l'application n'a pas terminé la procédure de validation »* : en Externe/Test, l'adresse n'est pas dans les utilisateurs de test.

2.5 Google Drive et Google Calendar

À quoi ça sert : Fichiers liste le dossier « Valemont OS » ; l'envoi d'un fichier se fait avec **ta** permission `drive.file` (le fichier t'appartient). L'Agenda fusionne l'agenda partagé « Valemont » et vos agendas ; l'agenda perso de l'autre n'apparaît qu'en « Occupé ». Les points fondateurs sont écrits dans l'agenda partagé.

Étapes Drive

1. Dans Google Drive, dossier « Valemont OS » › **Partager** › compte de service en **Lecteur**.
2. Copie l'identifiant dans l'URL `https://drive.google.com/drive/folders/<IDENTIFIANT>` et colle-le dans l'application : **Réglages › Espace de travail › Identifiant du dossier Drive**.

Étapes Calendar (Doc : https://support.google.com/calendar/answer/37082)

1. Google Agenda › ⚙ **Paramètres** › choisis l'agenda partagé « Valemont » › **Partager avec des personnes ou des groupes spécifiques** › ajoute le compte de service avec **Apporter des modifications aux événements**.
2. Chacun, pour son agenda personnel : même chemin, mais avec **Voir uniquement les disponibilités (masquer les détails)**.
3. Dans chaque agenda › **Intégrer l'agenda › ID de l'agenda** : copie-le dans **Réglages › Calendriers**.

Où ça va : dans l'application (Réglages › Espace de travail, Réglages › Calendriers) ; aucune variable.

Vérifier : Réglages › Intégrations › Google Drive et Google Calendar › **Tester la connexion** ; ouvrir Fichiers et Agenda. Premier envoi de fichier : Google te demande la permission Drive une fois (elle dure environ 1 h, puis un clic sur **Autoriser à nouveau**).

Problèmes fréquents : agenda vide = mauvais ID ou partage oublié ; fichier invisible = il n'est pas dans le dossier partagé.

2.6 Gmail (envoi seulement)

À quoi ça sert : les e-mails préparés par l'application (après un appel, relances, Croissance › Messages) partent de **ton** Gmail **quand tu cliques**. L'application ne lit jamais ta boîte de réception. Sans Gmail connecté, elle ouvre un brouillon Gmail pré-rempli : rien n'est bloqué.

Étapes

1. Vérifie (étape 3 du §2.4) que la Gmail API est activée, que `gmail.send` est dans l'Accès aux données et que l'URI `/api/auth/google/gmail/callback` est enregistrée.
2. Chacun, dans l'application : **Réglages › Intégrations › Gmail › Autoriser** (ou « Connecter Gmail » dans Appels). Google affiche l'écran de consentement : accepte.

Où ça va : rien à coller (même client OAuth).

Vérifier : la ligne Gmail montre l'autorisation active dans ce navigateur. Il n'y a **pas** de « Tester la connexion » pour Gmail (un test enverrait un vrai mail) : envoie-toi un e-mail de test depuis une fiche contact à ton adresse.

⏳ **Délais et limites Google** (Doc : https://developers.google.com/workspace/gmail/api/auth/scopes, https://support.google.com/cloud/answer/13463073)

- `gmail.send` est un champ d'application **sensible**.
- Application **Interne** (Workspace) : aucune vérification, rien à attendre. C'est le choix recommandé.
- Application **Externe en Test** : fonctionne pour vos deux adresses (écran « application non validée » à accepter), mais Google redemande le consentement souvent (au moins tous les 7 jours).
- Passer une application **Externe en production** avec `gmail.send` demande la **vérification Google** (vidéo de démonstration, politique de confidentialité, domaine vérifié) : compte **plusieurs jours à quelques semaines**.

2.7 Vercel : mise en ligne

À quoi ça sert : héberger l'application, garder les réglages de base, lancer les deux tâches planifiées.

Étapes

1. Clique sur le bouton **Deploy with Vercel** en haut du `README.md` (section « Deploy (LIVE, no terminal) »). Vercel copie le dépôt dans ton compte GitHub et affiche un formulaire qui ne demande que les réglages de base : `APP_MODE` (`live`), `SESSION_SECRET` (§2.1),

`FOUNDER_THEO_EMAIL`, `FOUNDER_PABLO_EMAIL`, `GOOGLE_OAUTH_CLIENT_ID`, `GOOGLE_OAUTH_CLIENT_SECRET` (§2.4), `SHEETS_SPREADSHEET_ID` (§2.3), `GOOGLE_SERVICE_ACCOUNT_KEY_BASE64` (le fichier JSON collé tel quel, §2.2). Clique **Deploy**. Si le projet Vercel existe déjà, ajoute plutôt ces variables dans **Settings › Environment Variables**. (Doc : https://vercel.com/docs/getting-started-with-vercel/import)

2. **Plan Pro** : le plan Hobby est réservé à un usage personnel non commercial et ne permet pas une tâche toutes les 6 h. Un seul membre payant suffit. (Doc : https://vercel.com/docs/limits/fair-use-guidelines, https://vercel.com/docs/cron-jobs/usage-and-pricing)
3. **Settings › Functions** : région **Paris (cdg1)** ; laisse le runtime Node.js et Fluid Compute (par défaut).
4. Ouvre `https://<votre-app>/setup`. Pour chaque réglage de base, la page dit s'il est défini, manquant, à corriger, automatique ou par défaut (jamais sa valeur), et où le trouver. Elle affiche les trois **URI de redirection** exactes à coller dans Google Auth Platform › Clients (§2.4).
5. Toujours sur `/setup`, **Générer un secret** → ajoute la valeur dans Vercel sous `CONNECTORS_ENCRYPTION_KEY` ; **Générer un secret** une seconde fois → `CRON_SECRET`. Dans **Settings › Environment Variables**, environnement **Production**, coche **Sensitive** (valeur ensuite illisible, même pour vous) et garde une copie dans le gestionnaire de mots de passe. (Doc : https://vercel.com/docs/environment-variables)
6. **Deployments › ··· › Redeploy**, puis **Vérifier à nouveau** sur `/setup` : tout doit être vert.
7. (Facultatif) **Settings › Domains** › ajoute `os.<domaine>` › crée chez ton registrar l'enregistrement DNS indiqué (CNAME) › attends le certificat. `APP_BASE_URL` suit automatiquement l'adresse de production de Vercel ; tu peux aussi la fixer (`https://os.<domaine>`, sans `/` final, puis Redeploy). Ajoute les 3 nouvelles URI de redirection chez Google. (Doc : https://vercel.com/docs/domains/working-with-domains/add-a-domain)
8. Tâches planifiées : rien à créer, `vercel.json` déclare la sauvegarde quotidienne (02:17 UTC) et la capture Instagram (toutes les 6 h). Vercel les appelle avec `CRON_SECRET`. Elles ne tournent que sur le déploiement de production. Visible dans **Settings › Cron Jobs**. (Doc : https://vercel.com/docs/cron-jobs/manage-cron-jobs#securing-cron-jobs)

Où ça va : Vercel. C'est la seule étape où l'on remplit Vercel ; tout le reste se fait dans l'application.

Ne jamais mettre en production : `SHEETS_API_BASE_URL`, `GOOGLE_API_BASE_URL`, `WISE_API_BASE_URL`, `STRIPE_API_BASE_URL`, `DEMO_BACKEND`, `FAKE_SHEETS_URL`, `FAKE_SHEETS_SPREADSHEET_ID`, `DEMO_ACCESS_CODE` (outils de test et de démo). `INSTAGRAM_API_BASE_URL` seulement dans le cas Instagram Login (§2.13).

Vérifier : `https://<votre-app>/api/health` répond `{"status":"ok"}` ou `"degraded"` (`fail` = un réglage de base manque ou le Sheet est inaccessible).

Problèmes fréquents : page « configuration requise » (`/setup`) = un réglage de base manque (le nom est affiché) ; variable ajoutée dans Vercel mais rien ne change = pas redéployé.

2.8 Premier démarrage (Phase 3)

Pas de terminal : la base se prépare depuis l'application.

1. Ouvre l'application et clique sur **Se connecter avec Google** (Théo ou Pablo, peu importe qui commence).
2. Tant que le Sheet n'est pas prêt, tu arrives sur la page **Installation**. Clique sur **Préparer la base de données** : l'application crée les onglets et les en-têtes, fige les en-têtes, ajoute une protection d'avertissement et les réglages de départ, puis lance la vérification. L'avancement s'affiche, avec le résultat onglet par onglet. Sans danger à relancer : elle n'efface et n'écrase jamais rien.
3. Si le compte de service n'a pas l'accès **Éditeur**, la page le dit et affiche l'adresse exacte du compte de service : Partager › colle l'adresse › Éditeur › décoche la notification › Partager, puis reclique.
4. Ton Sheet privé (si `PRIVATE_SHEET_ID_<prénom>` est défini) est préparé en même temps ; chacun prépare le sien à sa première connexion (l'application ne touche jamais au Sheet privé de l'autre).
5. Ensuite, Ma journée s'ouvre. Pablo se connecte dans un autre navigateur ; un compte non fondateur est refusé. Remplissez Réglages › Calendriers et Réglages › Espace de travail. **Réglages › Diagnostic** : santé « prête », et **Vérifier** (vérification de la base de données) quand vous voulez.
6. Après une mise à jour qui ajoute des onglets ou des colonnes, la page Installation revient une fois : un clic.

Avancé (facultatif) : `npm run setup:sheets` et `npm run doctor:sheets` font la même chose depuis un terminal (voir `docs/DEPLOYMENT.md` §5).

2.9 Wise (banque, lecture seule)

À quoi ça sert : page Finances et trésorerie du Command Center : soldes, relevés, mouvements en direct par webhook. Le même jeton en lecture seule donne aussi les **taux USD→EUR de Wise** utilisés pour les **paiements fournisseurs en dollars**. **Le jeton ne peut jamais déplacer d'argent.**

Ce qu'il te faut : accès administrateur au compte **Wise Business**, ton téléphone (double authentification).

Étapes (Doc : https://docs.wise.com/guides/developer/auth-and-security/personal-api-token)

1. Dans l'application : **Réglages › Intégrations › Wise › Configurer**. Garde la fenêtre ouverte.
2. wise.com › compte Business › **Paramètres › API tokens** (selon la version : *Connect and manage apps › API tokens* ou *Integrations and Tools › API tokens*) › **Add new token** › nom « Valemont OS » › accès **Read only** › valide la double authentification. Le jeton n'est affiché **qu'une fois** : colle-le directement dans le champ **Jeton d'API en lecture seule** (`WISE_API_TOKEN` , forme 8-4-4-4-12 lettres et chiffres).
3. **Identifiant du profil entreprise** : laisse vide, l'application trouve le profil Business toute seule.

4. (Facultatif, mouvements en direct) Chez Wise, ajoute un webhook `balances#update` avec l'**adresse du webhook** affichée dans le formulaire (`https://<votre-app>/api/webhooks/wise`), dans Wise › Paramètres › **Webhooks** si le menu existe sur ton compte (Doc : https://docs.wise.com/guides/developer/webhooks/subscribe-to-webhooks). Puis copie la **clé publique de production** des webhooks Wise (page « Event handling » : https://docs.wise.com/guides/developer/webhooks/event-handling) et colle-la **entière** (lignes BEGIN et END comprises) dans **Clé publique des webhooks Wise**. Pas besoin de la convertir : l'application s'en charge. Si le menu Webhooks n'existe pas, saute cette étape : le bouton **Synchroniser** rattrape les mouvements.
5. **Enregistrer et tester** : l'application lit tes soldes une fois. Vert = c'est branché.

Avancé (facultatif) : **clé SCA** pour lire les relevés quand Wise exige une authentification forte. Elle se fabrique en terminal (`openssl genrsa -out wise-private.pem 2048` , puis `openssl rsa -pubout -in wise-private.pem -out wise-public.pem`) ; dépose `wise-public.pem` dans Wise › API tokens › **Manage public keys**, colle le contenu de `wise-private.pem` dans **Clé privée pour les relevés (SCA)**, puis supprime les fichiers. Dans l'UE/au Royaume-Uni (DSP2), Wise peut refuser les relevés via un jeton personnel, même avec la clé : les soldes et le webhook marchent quand même, et l'historique vient d'un **export CSV de relevé Wise** (Finances › **Importer un CSV Wise**). Créer l'abonnement webhook par l'API est aussi possible (voir `docs/INTEGRATIONS.md`).

Où ça va : dans l'app (Configurer) : `WISE_API_TOKEN` (obligatoire pour Wise), `WISE_PROFILE_ID` , `WISE_WEBHOOK_PUBLIC_KEY_BASE64` , `WISE_SCA_PRIVATE_KEY_BASE64` (facultatives).

Vérifier : Enregistrer et tester en vert, puis **Synchroniser** (relevés). Les soldes apparaissent dans Finances. Un crédit classé automatiquement en revenu reste « à vérifier » tant qu'un fondateur ne l'a pas confirmé.

Problèmes fréquents : rouge 401 = jeton révoqué ou mal copié (recolle-le) ; « SCA requise » = clé SCA absente ou refusée → import CSV ; mauvais profil = renseigne l'identifiant du profil entreprise dans le formulaire.

2.10 Stripe (lecture seule)

À quoi ça sert : solde Stripe, ventes, frais, virements, MRR (Finances et Command Center). La clé ne peut ni encaisser ni rembourser. Les virements Stripe → Wise sont des transferts internes : une vente n'est jamais comptée deux fois.

Étapes (Doc : https://docs.stripe.com/keys/restricted-api-keys, https://docs.stripe.com/webhooks)

1. Dans l'application : **Réglages › Intégrations › Stripe › Configurer**.
2. Dashboard Stripe (mode **live**) › **Développeurs › Clés API › Créer une clé restreinte** › nom « Valemont OS ». Mets **Lecture** sur : *Balance*, *Balance transactions*, *Subscriptions*, *Customers*. Tout le reste : **Aucun**. Colle la clé (`rk_live_…`) dans **Clé restreinte (lecture seule)**. Une clé `pk_` est refusée.

3. **Développeurs › Webhooks** (dans Workbench : *Destinations d'événements*) › **Ajouter une destination** › colle l'**adresse du webhook** affichée dans le formulaire (`https://<votre-app>/api/webhooks/stripe`) › événements : `charge.*` , `payout.*` , `customer.subscription.*` , `invoice.paid` , `balance.available` .
4. Ouvre la destination › **Secret de signature** › Révéler (`whsec_…`) › colle-le dans **Secret de signature du webhook**.
5. **Enregistrer et tester** : l'application lit le solde une fois.

Où ça va : dans l'app (Configurer) : `STRIPE_API_KEY` (obligatoire pour Stripe), `STRIPE_WEBHOOK_SECRET` (facultative mais recommandée).

Vérifier : vert après **Enregistrer et tester**, puis **Synchroniser** ; dans Stripe, *Envoyer un événement de test* sur la destination doit répondre 200.

Problèmes fréquents : 403 « permission manquante » = une ressource n'est pas en Lecture (le message dit laquelle) ; 400 sur le webhook = secret d'une autre destination ou du mode test.

2.11 Site web : formulaires → webhook signé

À quoi ça sert : chaque formulaire (contact, prospect, réservation) des sites FR/ES crée un prospect entrant (propriétaire Théo pour FR, Pablo pour ES) et un élément « À répondre ». Un contact désinscrit est bloqué : rien n'est créé.

Étapes

1. Dans l'application : **Réglages › Intégrations › Formulaires du site › Configurer** › **Générer** à côté de **Secret de signature des formulaires**. Le secret (`WEBSITE_WEBHOOK_SECRET`) est enregistré chiffré et s'affiche **une seule fois** : copie-le.
2. Chez l'hébergeur du site (Netlify › Site configuration › Environment variables), ajoute `WEBSITE_WEBHOOK_SECRET` avec cette valeur et `VALEMONT_WEBHOOK_URL` avec l'**adresse du webhook des formulaires** affichée dans le formulaire (`https://<votre-app>/api/webhooks/website`).
3. Netlify Forms : le webhook sortant de Netlify ne sait pas signer, il faut une petite fonction **submission-created** (code prêt à copier dans `docs/INTEGRATIONS.md` › « Netlify Forms »). Doc : https://docs.netlify.com/functions/trigger-on-events/. Autre hébergeur : n'importe quel code côté serveur (Worker Cloudflare, Apps Script…) qui signe de la même façon (`docs/INTEGRATIONS.md` › « Generic form handler »). C'est la seule étape « code » : la personne qui gère le site la fait une fois. **Le secret ne doit jamais être dans le code du navigateur.**
4. Dans le formulaire : champ piège `bot-field` , champs cachés `lang` , `page` , UTM, `consent_version` , et une case de consentement dont la phrase est envoyée en `consent_text` .
5. **Enregistrer et tester**, puis envoie un vrai formulaire de test.

Où ça va : dans l'app (Configurer) : `WEBSITE_WEBHOOK_SECRET` ; côté site (pas dans Valemont OS) : la même valeur + `VALEMONT_WEBHOOK_URL` .

Vérifier : vert après **Enregistrer et tester** (le secret est présent et assez long) ; le formulaire de test apparaît dans Ma journée › À répondre.

Problèmes fréquents : 401 `stale_timestamp` = horloge du serveur décalée de plus de 5 min ; 401 `bad_signature` = secrets différents (après un nouveau **Générer**, recopie-le chez l'hébergeur) ou signature calculée sur un JSON reformaté (il faut signer le texte exact envoyé).

2.12 Messages à traiter : relais Make / Zapier / n8n et Instagram

À quoi ça sert : rassembler ce qui attend une réponse (commentaires, DM, notifications LinkedIn…) avec une réponse suggérée dans ta langue. **Rien n'est jamais envoyé automatiquement** : « Copier une réponse » ouvre la conversation et tu colles toi-même. Les réponses e-mail/WhatsApp marquées et les prospects du site arrivent déjà sans rien brancher.

Tout se règle dans **Réglages › Intégrations › Messages à traiter › Configurer**.

A. Relais (Make, Zapier ou n8n)

1. **Générer** à côté de **Secret de signature du relais** (`INBOX_WEBHOOK_SECRET`) : copie la valeur affichée une seule fois et colle-la dans le relais (variable / connexion protégée).
2. Déclencheur au choix : « Instagram for Business › Watch comments », boîte mail du relais (Make *Mailhook*, *Email by Zapier*) alimentée par un **filtre Gmail** qui transfère les notifications LinkedIn, etc.
3. Action : HTTP POST vers l'**adresse du relais** affichée dans le formulaire (`https://<votre-app>/api/inbox/webhook`), corps JSON (format dans `docs/INBOX.md`), en-tête `X-Valemont-Signature: sha256=<HMAC-SHA256 du corps exact avec le secret>` : Make = fonction `sha256(body; "hex"; secret)` en mode HMAC ; Zapier = étape *Code by Zapier* ; n8n = nœud *Crypto* (HMAC, SHA256, hex). `external_id` doit être stable (c'est l'anti-doublon).

B. Instagram via l'app Meta (voir aussi §2.13 pour l'app elle-même)

1. Dans le formulaire, **Générer** à côté de **Jeton de vérification Meta** (`META_VERIFY_TOKEN`) : il est déjà enregistré ; copie-le.
2. Dans l'app Meta › **Webhooks** (ou Instagram › *API setup* › *Configure webhooks*) : URL de rappel = l'**adresse Meta** du formulaire (`https://<votre-app>/api/inbox/meta`), jeton de vérification = la valeur copiée › **Vérifier et enregistrer**. (Doc : https://developers.facebook.com/docs/instagram-platform/webhooks)
3. Abonne les champs `comments` et `messages` pour les deux comptes (Valemont, 30/30).
4. **Paramètres de l'app › Général › Clé secrète** (32 lettres et chiffres) › colle-la dans **Clé secrète de l'app Meta** (`META_APP_SECRET`) › **Enregistrer et tester**.

5. (Facultatif) Nommer les comptes : réglage `instagram_account_labels` (voir `docs/INBOX.md`).
6. ⏳ Permissions `instagram_business_manage_comments`, `instagram_business_manage_messages` (ou équivalents `instagram_manage_comments` / `instagram_manage_messages`) : pour recevoir les événements de comptes qui n'ont pas de rôle dans l'app, Meta exige l'**App Review** (quelques jours à quelques semaines, vidéo de démonstration, politique de confidentialité). En attendant, utilisez le relais Make « Watch comments ». (Doc : https://developers.facebook.com/docs/resp-plat-initiatives/individual-processes/app-review)

Où ça va : dans l'app (Configurer) : `INBOX_WEBHOOK_SECRET`, `META_VERIFY_TOKEN` (générés ici), `META_APP_SECRET` (collé depuis Meta).

Vérifier : vert après **Enregistrer et tester** ; puis un commentaire de test sur Instagram (ou un appel du relais) apparaît dans Ma journée › À répondre.

Problèmes fréquents : Meta n'arrive pas à « vérifier et enregistrer » l'URL = jeton collé chez Meta différent de celui généré (régénère-le et recopie-le) ; 401 du relais = signature calculée sur un autre texte que le corps envoyé, ou ancien secret encore dans le relais.

2.13 Instagram : statistiques (lecture seule) et capture toutes les 6 h

À quoi ça sert : abonnés, vues, portée, visites de profil, clics vers le site, et chiffres de chaque post/Reel à 24 h, 72 h, 7 j, 30 j, stories toutes les 6 h (Studio contenu, Croissance). Sans jeton : saisie à la main, marquée « saisi à la main ».

Ce qu'il te faut : chaque compte Instagram en **compte professionnel** (Business ou Créateur) **relié à une Page Facebook** (Doc : https://developers.facebook.com/docs/instagram-platform/overview), le tout dans le **portefeuille business Meta de Valemont**.

Étapes (méthode recommandée : Facebook Login + utilisateur système, jeton sans expiration)

1. Dans l'application : **Réglages › Intégrations › Instagram › Configurer**.
2. https://developers.facebook.com › **Mes apps › Créer une app** › type Business, rattachée au portefeuille Valemont (jamais un compte personnel tiers). Ajoute le produit **Instagram** (API avec connexion Facebook). (Doc : https://developers.facebook.com/docs/instagram-platform/instagram-api-with-facebook-login/)
3. **Paramètres business** (business.facebook.com/settings) › **Utilisateurs › Utilisateurs système › Ajouter** (rôle admin) › **Attribuer des éléments** : l'app, la Page et les comptes Instagram. (Doc : https://developers.facebook.com/docs/marketing-api/system-users/)
4. **Générer un jeton** › app Valemont OS › expiration **Jamais** › autorisations `instagram_basic`, `instagram_manage_insights`, `pages_show_list`, `pages_read_engagement` › colle-le

dans **Jeton d'accès** (`INSTAGRAM_ACCESS_TOKEN`).

5. Identifiant du compte : dans l'Explorateur de l'API Graph, `me/accounts?fields=instagram_business_account` → le nombre `instagram_business_account.id` › colle-le dans **Identifiant du compte Instagram** (`INSTAGRAM_BUSINESS_ACCOUNT_ID`).
6. **Quel compte** : laisse vide pour `ig-valemont` , ou mets `ig-thirty` pour 30/30. **Date d'expiration** : vide pour un jeton d'utilisateur système.
7. **Enregistrer et tester** : l'application lit le profil une fois. Accès Standard sur vos propres comptes : **pas d'App Review nécessaire** pour les statistiques.

Avancé (facultatif) : **deux comptes en même temps** = variable `INSTAGRAM_ACCOUNTS_JSON` dans Vercel (format exact dans `.env.example` , puis Redeploy), à la place du formulaire. **Méthode de secours (Instagram Login, jeton de 60 jours)** : permissions `instagram_business_basic` + `instagram_business_manage_insights` , variable Vercel `INSTAGRAM_API_BASE_URL=https://graph.instagram.com/v21.0` , et la date d'expiration dans le formulaire. L'application affiche les jours restants et crée une tâche 14 jours avant ; renouvellement : https://developers.facebook.com/docs/instagram-platform/reference/refresh_access_token, puis colle le nouveau jeton dans **Configurer** (pas de redéploiement).

Capture toutes les 6 h : déjà planifiée par Vercel Cron (`CRON_SECRET` , Phase 2). La ligne **Capture planifiée › Configurer** ne sert que si un autre planificateur l'appelle (Apps Script, GitHub Actions, cron-job.org) : **Générer** le jeton de la tâche, copie-le une seule fois dans le planificateur, qui appelle l'**adresse de la tâche** affichée toutes les 6 h avec l'en-tête `Authorization: Bearer <jeton>` (ou génère plutôt le secret de signature si le planificateur sait signer, voir `docs/INTEGRATIONS.md`) › **Enregistrer**.

Où ça va : dans l'app (Configurer) : `INSTAGRAM_ACCESS_TOKEN` , `INSTAGRAM_BUSINESS_ACCOUNT_ID` , `INSTAGRAM_ACCOUNT_KEY` , `INSTAGRAM_TOKEN_EXPIRES_AT` , `SNAPSHOT_JOB_TOKEN` , `SNAPSHOT_JOB_SECRET` ; Vercel (avancé) : `INSTAGRAM_ACCOUNTS_JSON` , `INSTAGRAM_API_BASE_URL` .

Vérifier : vert après **Enregistrer et tester**, puis **Synchroniser** ; la ligne « Capture planifiée » montre la dernière exécution ; Studio contenu affiche les chiffres.

Problèmes fréquents : pas d' `instagram_business_account` = compte non professionnel ou non relié à la Page ; statistiques de story « non disponible » sous 5 spectateurs (c'est normal, jamais 0) ; démographie absente sous 100 abonnés.

2.14 LinkedIn (imports, et API plus tard)

À quoi ça sert : impressions, abonnés et pays des Pages Valemont / 30/30 et de vos profils créateurs, dans Croissance. La prospection LinkedIn reste manuelle (journal + ouverture du profil) : pas de scraping, pas de message automatique.

Maintenant : les exports

1. Page LinkedIn (en tant qu'admin) › **Analytics** › onglet (Contenu, Visiteurs, Abonnés) › **Exporter** › période › fichier **.xls**.
2. Profil créateur : **Analytics** de ton profil › **Exporter** › fichier **.xlsx**.
3. Dans l'application : Croissance › **Ajouter des chiffres** › **Importer un export LinkedIn**. Réimporter le même fichier n'ajoute rien en double. Une fois par semaine suffit.

Plus tard (⏳ facultatif) : API Community Management

- Créer une **nouvelle** app LinkedIn Developers dédiée (aucun autre produit), reliée à la **Page vérifiée** de Valemont, demander l'accès *Development tier*, puis le *Standard tier* avec une vidéo de chaque usage. Examen long, refus possible (il faut alors une nouvelle app). L'adaptateur ne sera construit qu'après approbation. (Doc : https://learn.microsoft.com/en-us/linkedin/marketing/community-management/community-management-overview, https://learn.microsoft.com/en-us/linkedin/marketing/increasing-access)

Où ça va : rien à coller. **Statut attendu :** *En attente d'approbation* (normal).

2.15 GA4 et Search Console (imports CSV)

À quoi ça sert : visites du site et prospects par source (dont les moteurs d'IA) dans Croissance ; requêtes et pages (clics, impressions, position) dans Croissance › SEO et l'optimiseur du Studio contenu.

GA4 (Doc : https://support.google.com/analytics/answer/9317657)

1. analytics.google.com › **Rapports** › Acquisition › Acquisition de trafic › période (ex. 7 derniers jours).
2. **Partager ce rapport** (en haut à droite) › **Télécharger le fichier** › **CSV**.
3. Croissance › **Ajouter des chiffres** › importer le CSV GA4.

Search Console (Doc : https://support.google.com/webmasters/answer/7576553)

1. search.google.com/search-console › propriété du site › **Performances** › période.
2. **Exporter** › **Télécharger le fichier CSV**.
3. Croissance › SEO › **Ajouter des chiffres Search Console**.

Où ça va : rien à coller. **Statut attendu :** *Import uniquement* avec la date du dernier import. Routine conseillée : le lundi, les deux exports.

2.16 Mon tracker (ton app de suivi personnelle)

À quoi ça sert : chacune de tes activités (appels, e-mails, visites…) est envoyée à ton tracker personnel, signée ; ton tracker peut aussi lire tes activités via un flux. Seulement en LIVE. Chacun configure **les siennes**, connecté avec son propre compte : l'autre ne voit ni ne modifie tes valeurs.

Étapes (dans **Réglages › Intégrations › Mon tracker › Configurer**)

1. Dans ton app de suivi, crée un **webhook entrant** qui accepte du JSON et copie son adresse (https obligatoire) › colle-la dans **Adresse du tracker** (`TRACKER_WEBHOOK_URL_<prénom>`).
2. **Générer** à côté de **Secret de signature** (`TRACKER_WEBHOOK_SECRET_<prénom>`) : copie la valeur affichée une seule fois et donne-la à ton tracker pour qu'il vérifie l'en-tête `X-Valemont-Signature` (code de vérification dans `docs/TRACKER.md`).
3. (Facultatif) Flux : **Générer** le **jeton du flux** (`TRACKER_FEED_TOKEN_<prénom>`) ; ton tracker appelle l'**adresse du flux** affichée (`https://<votre-app>/api/tracker/feed?from=AAAA-MM-JJ&to=AAAA-MM-JJ`) avec `Authorization: Bearer <jeton>` .
4. **Enregistrer et tester** : un événement de test signé part vers ton tracker.

Avancé (facultatif) : `TRACKER_DAILY_SUMMARY_THEO=1` (ou `_PABLO`) dans Vercel pour envoyer aussi le résumé du jour à l'enregistrement du bilan.

Où ça va : dans l'app (Configurer, chacun les siennes) : `TRACKER_WEBHOOK_URL_*` , `TRACKER_WEBHOOK_SECRET_*` , `TRACKER_FEED_TOKEN_*` ; Vercel (avancé) : `TRACKER_DAILY_SUMMARY_*` .

Vérifier : vert après **Enregistrer et tester** (ou Réglages › **Mon tracker** › **Envoyer un événement de test**) ; « Envoyer cette période » rattrape un trou sans doublon.

Problèmes fréquents : adresse en `http://` refusée ; le tracker doit répondre 2xx en moins de 5 s.

2.17 Sauvegardes, tâches planifiées, restauration

À quoi ça sert : une copie Google Sheet quotidienne, vérifiée, gardée 30 jours ; une restauration testée.

Étapes

1. Dossier de sauvegarde :
   - **Google Workspace** : crée un **Drive partagé** « Valemont OS – sauvegardes », ajoute le compte de service en **Gestionnaire de contenu** ;
   - **compte Google personnel** : un compte de service n'a pas d'espace de stockage. Crée un dossier normal, partage-le en Éditeur, et fais la copie avec le petit **Apps Script** à toi (code à copier-coller et déclencheur quotidien dans `docs/DEPLOYMENT.md` › 6. Backups, tout se fait dans le navigateur).
2. Dans l'application : **Réglages › Intégrations › Sauvegardes › Configurer** › colle l'**adresse du dossier** (ou seulement son identifiant) dans **Dossier des sauvegardes** (`BACKUP_DRIVE_FOLDER_ID`). **Jours à conserver** : vide = 30 (1 à 365).
3. Planification : rien à faire, Vercel Cron lance la sauvegarde à 02:17 UTC avec `CRON_SECRET` (Phase 2). Sans planificateur, la première visite de la journée lance la copie. (Facultatif) Pour un autre planificateur : **Générer** le **jeton de la tâche** et copie-le une seule fois chez lui.
4. **Enregistrer et tester** : l'application liste le dossier une fois.

5. **Exercice de restauration** (le lendemain, puis chaque trimestre), sans terminal : Réglages › Diagnostic › **Sauvegardes** › **Vérifier** sur la copie du jour (elle est comparée à la base actuelle : onglets, en-têtes, nombre de lignes). En cas de vrai besoin : ouvre la copie › Fichier › **Créer une copie**, partage-la avec le compte de service en Éditeur, mets son identifiant dans `SHEETS_SPREADSHEET_ID` (Vercel) › Redeploy ; la page Installation vérifie tout. **Ne bascule pas la production** pour un exercice, et garde l'ancien Sheet intact.
6. (Facultatif) Un moniteur de disponibilité (UptimeRobot, Better Stack…) sur `https://<votre-app>/api/health`, une fois par minute au plus.

Avancé (facultatif) : restauration complète en terminal dans un **nouveau** Sheet vide : `npm run restore:sheets -- --from-sheet <id de la copie> --to <id du nouveau Sheet>`, puis `SHEETS_SPREADSHEET_ID=<nouvel id> npm run doctor:sheets` (le script refuse le Sheet de production et tout Sheet non vide ; détails dans `docs/DEPLOYMENT.md` §7). `BACKUP_JOB_SECRET` (signature) se met dans Vercel.

Où ça va : dans l'app (Configurer) : `BACKUP_DRIVE_FOLDER_ID`, `BACKUP_RETENTION_DAYS`, `BACKUP_JOB_TOKEN` ; Vercel : `CRON_SECRET` (Phase 2), `BACKUP_JOB_SECRET` (avancé).

Vérifier : Réglages › Diagnostic › **Sauvegardes** : « Valemont OS backup » présente et **vérifiée** ; bouton **Vérifier** pour comparer à la base actuelle.

Problèmes fréquents : `storageQuotaExceeded` = compte personnel sans Drive partagé → solution Apps Script ; rouge au test = dossier pas partagé avec le compte de service.

2.18 Vérifications finales (Phase 10)

1. Réglages › Intégrations : clique **Tester la connexion** sur chaque ligne configurée → **Connecté**.
2. Réglages › Diagnostic : santé « prête », sauvegarde vérifiée.
3. Dérouler la checklist « Stage 4 gate: live validation » de `docs/ACCEPTANCE.md` à deux (connexion des deux, appel vu par l'autre en moins d'une minute, conflit d'édition, `+33…` intact dans le Sheet, agenda, fichiers, imports LinkedIn, sauvegarde, restauration, `/api/health`, en-têtes de sécurité).
4. Noter le résultat daté dans `docs/STATUS.md`. Une intégration n'est **VERIFIED_LIVE** qu'après ce test réel.

---

3. Toutes les variables

Deux endroits :

- **Vercel (base)** : **Vercel › Settings › Environment Variables** (Production, *Sensitive* pour les secrets), puis **Redeploy**. Ce sont les réglages dont l'application a besoin avant de pouvoir ouvrir le Sheet ; Réglages › Intégrations les montre en lecture seule.
- **Dans l'app (Configurer)** : **Réglages › Intégrations › Configurer** ; pris en compte tout de suite, sans redéploiement. Une variable du même nom dans Vercel reste possible et **a priorité**.

« Vous » = une valeur que vous générez vous-mêmes (§2.1) ; « Générer » = le bouton du formulaire.

Variable	Où la mettre	Obligatoire ?	Où la trouver	Intégration
`APP_MODE`	Vercel (base)	Obligatoire (`live`)	Vous	Base
`SESSION_SECRET`	Vercel (base)	Obligatoire	Vous (32+ car., `/setup` › Générer un secret)	Base
`CONNECTORS_ENCRYPTION_KEY`	Vercel (base)	Fortement recommandée	`/setup` › Générer un secret	Chiffrement des clés de l'app
`CONNECTORS_ENCRYPTION_KEY_PREVIOUS`	Vercel (base)	Seulement pendant un changement de clé	L'ancienne clé	Chiffrement des clés de l'app
`APP_BASE_URL`	Vercel (base)	Facultative sur Vercel (automatique), obligatoire ailleurs	Votre domaine, `https://…` sans `/` final	Base, Sign-In, webhooks
`SYNC_POLL_SECONDS`	Vercel (base)	Facultative (30)	30 à 60	Base
`LOG_LEVEL`	Vercel (base)	Facultative (`info`)	`debug` / `info` / `warn` / `error` / `silent`	Journaux
`FOUNDER_THEO_EMAIL` , `FOUNDER_PABLO_EMAIL`	Vercel (base)	Obligatoires	Vos adresses Google	Sign-In
`FOUNDER_THEO_TIMEZONE` , `FOUNDER_PABLO_TIMEZONE`	Vercel (base)	Facultatives (défaut `Europe/Paris` , `Europe/Madrid`)	`Europe/Paris` , `Europe/Madrid`	Base
`GOOGLE_OAUTH_CLIENT_ID` , `GOOGLE_OAUTH_CLIENT_SECRET`	Vercel (base)	Obligatoires	Google Auth Platform › Clients	Sign-In, Drive, Gmail
`SHEETS_SPREADSHEET_ID`	Vercel (base)	Obligatoire	URL du Sheet	Google Sheets
`GOOGLE_SERVICE_ACCOUNT_KEY_BASE64`	Vercel (base)	Obligatoire	Clé JSON du compte de service, collée telle quelle (ou en base64)	Sheets, Drive, Calendar, sauvegardes
`PRIVATE_SHEET_ID_THEO` , `PRIVATE_SHEET_ID_PABLO`	Vercel (base)	Facultatives	URL du Sheet privé de chacun	Google Sheets (privé)
`CRON_SECRET`	Vercel (base)	Recommandée sur Vercel	`/setup` › Générer un secret (24+ car.)	Sauvegardes + capture
`WISE_API_TOKEN`	Dans l'app (Configurer)	Obligatoire pour Wise	Wise › Paramètres › API tokens (Read only)	Wise
`WISE_PROFILE_ID`	Dans l'app (Configurer)	Facultative (trouvé tout seul)	Profil BUSINESS	Wise

Variable	Où la mettre	Obligatoire ?	Où la trouver	Intégration
`WISE_WEBHOOK_PUBLIC_KEY_BASE64`	Dans l'app (Configurer)	Facultative	Clé publique de production des webhooks Wise, collée entière	Wise
`WISE_SCA_PRIVATE_KEY_BASE64`	Dans l'app (Configurer)	Facultative (avancé)	Votre clé RSA privée (publique déposée chez Wise), collée entière	Wise
`STRIPE_API_KEY`	Dans l'app (Configurer)	Obligatoire pour Stripe	Stripe › Développeurs › Clés API › clé restreinte `rk_live_…`	Stripe
`STRIPE_WEBHOOK_SECRET`	Dans l'app (Configurer)	Facultative (recommandée)	Stripe › Webhooks › destination › secret `whsec_…`	Stripe
`WEBSITE_WEBHOOK_SECRET`	Dans l'app (Configurer)	Obligatoire pour les formulaires	Générer, puis aussi côté site	Formulaires du site
`INBOX_WEBHOOK_SECRET`	Dans l'app (Configurer)	Obligatoire pour le relais	Générer, puis aussi dans Make/Zapier/n8n	Messages à traiter
`META_APP_SECRET`	Dans l'app (Configurer)	Facultative	App Meta › Paramètres › Général	Messages (Instagram)
`META_VERIFY_TOKEN`	Dans l'app (Configurer)	Facultative	Générer, puis aussi dans les Webhooks Meta	Messages (Instagram)
`INSTAGRAM_ACCESS_TOKEN`	Dans l'app (Configurer)	Obligatoire pour 1 compte	Utilisateur système Meta (ou Instagram Login)	Instagram
`INSTAGRAM_BUSINESS_ACCOUNT_ID`	Dans l'app (Configurer)	Obligatoire pour 1 compte	Explorateur Graph API	Instagram
`INSTAGRAM_ACCOUNT_KEY`	Dans l'app (Configurer)	Facultative	`ig-valemont` (défaut) ou `ig-thirty`	Instagram
`INSTAGRAM_TOKEN_EXPIRES_AT`	Dans l'app (Configurer)	Facultative	Date d'expiration d'un jeton 60 jours	Instagram
`INSTAGRAM_ACCOUNTS_JSON`	Vercel (avancé)	Facultative (à la place des 4 ci-dessus)	Vous (liste JSON, voir `.env.example`)	Instagram
`INSTAGRAM_API_BASE_URL`	Vercel (avancé)	Facultative	Seulement `https://graph.instagram.com/v21.0` en Instagram Login	Instagram

Variable	Où la mettre	Obligatoire ?	Où la trouver	Intégration
`SNAPSHOT_JOB_TOKEN`, `SNAPSHOT_JOB_SECRET`	Dans l'app (Configurer)	Facultatives	Générer (si planificateur autre que Vercel)	Capture planifiée
`TRACKER_WEBHOOK_URL_THEO` / `_PABLO`	Dans l'app (Configurer, chacun le sien)	Facultatives	Ton app de suivi (https)	Mon tracker
`TRACKER_WEBHOOK_SECRET_THEO` / `_PABLO`	Dans l'app (Configurer, chacun le sien)	Obligatoire avec l'adresse	Générer	Mon tracker
`TRACKER_FEED_TOKEN_THEO` / `_PABLO`	Dans l'app (Configurer, chacun le sien)	Facultatives	Générer	Mon tracker
`TRACKER_DAILY_SUMMARY_THEO` / `_PABLO`	Vercel (avancé)	Facultatives	`1` pour activer	Mon tracker
`BACKUP_DRIVE_FOLDER_ID`	Dans l'app (Configurer)	Facultative (recommandée)	Adresse du dossier / Drive partagé de sauvegarde	Sauvegardes
`BACKUP_RETENTION_DAYS`	Dans l'app (Configurer)	Facultative (30)	1 à 365	Sauvegardes
`BACKUP_JOB_TOKEN`	Dans l'app (Configurer)	Facultative	Générer (si planificateur autre que Vercel)	Sauvegardes
`BACKUP_JOB_SECRET`	Vercel (avancé)	Facultative	Vous (si planificateur qui signe)	Sauvegardes
`SHEETS_API_BASE_URL`, `GOOGLE_API_BASE_URL`, `WISE_API_BASE_URL`, `STRIPE_API_BASE_URL`, `DEMO_BACKEND`, `FAKE_SHEETS_URL`, `FAKE_SHEETS_SPREADSHEET_ID`, `DEMO_ACCESS_CODE`	**Jamais en production**	—	Tests et démo uniquement	—

Côté site (pas dans Valemont OS) : `WEBSITE_WEBHOOK_SECRET` (même valeur) et `VALEMONT_WEBHOOK_URL`. Les identifiants d'agendas et du dossier Drive se saisissent **dans l'application** (Réglages › Calendriers, Réglages › Espace de travail), pas en variables.

---

4. Rappels de sécurité

- **Les clés collées dans l'application sont chiffrées** (AES-256-GCM) avant d'être enregistrées : le Sheet ne contient qu'un texte illisible (onglet `IntegrationSecrets`), et la clé de chiffrement (`CONNECTORS_ENCRYPTION_KEY`) reste dans Vercel, jamais dans le Sheet. Les identifiants non secrets (profil Wise, compte Instagram, dossier de sauvegarde) sont enregistrés en clair dans les réglages.
- **Une clé enregistrée n'est plus jamais affichée** : seulement « Configuré · se termine par 1234 », qui l'a enregistrée et quand. Pour la changer, colle une nouvelle valeur ou clique sur **Générer** à nouveau ; **Supprimer** l'efface. Un secret généré s'affiche une seule fois : copie-le tout de suite dans l'autre service.
- **Seuls Théo et Pablo peuvent modifier les clés** (connectés avec leur compte Google de fondateur). Chaque changement est inscrit au journal d'audit (le nom du champ seulement, jamais la valeur). Les valeurs du tracker sont propres à chacun.
- **Les réglages de base vivent uniquement dans Vercel** (cochés *Sensitive*) et dans votre gestionnaire de mots de passe. Aucun secret dans un chat, un mail, directement dans le Sheet, un ticket, une capture d'écran ou le code. Les journaux masquent clés, e-mails et numéros.
- **Moindre privilège partout :** compte de service sans rôle (seulement les partages utiles), dossier Drive en Lecteur, agendas perso en « disponibilités seulement », Wise en **lecture seule**, Stripe en **clé restreinte Lecture**, Instagram en statistiques/lecture, Gmail en **envoi seulement**, Drive en `drive.file` (seulement les fichiers créés par l'app).
- **Ce que l'application ne fait jamais :**
  - envoyer un e-mail, un WhatsApp, une réponse Instagram ou LinkedIn sans **ton clic** ;
  - lire ta boîte Gmail ;
  - déplacer de l'argent, payer, rembourser (Wise et Stripe en lecture seule) ;
  - contacter un contact désinscrit (blocage côté serveur, pour vous deux) ;
  - montrer à l'un les données privées de l'autre (tâches privées, détails d'agenda perso) ;
  - utiliser la démo à la place des vraies données : sans réglages de base, le mode LIVE affiche une erreur de configuration (en démo, rien de ce que tu colles n'est vraiment enregistré).
- **Rotation :**
  - fuite ou doute sur une clé d'un service → la régénérer chez le fournisseur, puis **Configurer** › coller la nouvelle valeur › **Enregistrer et tester** (pas de redéploiement) ; pour un secret que l'on crée, **Générer** à nouveau et le recopier dans l'autre service ;
  - clé Google → nouvelle clé, variable Vercel, Redeploy, **puis** suppression de l'ancienne ;
  - `SESSION_SECRET` changé = tout le monde est déconnecté (c'est la façon de révoquer une session volée). Sans `CONNECTORS_ENCRYPTION_KEY`, les clés enregistrées dans l'app deviennent aussi illisibles (« illisible, colle-la de nouveau ») : c'est pour ça qu'on définit cette clé dès la Phase 2 ;

    - changer `CONNECTORS_ENCRYPTION_KEY` : mets la nouvelle valeur, l'ancienne dans `CONNECTORS_ENCRYPTION_KEY_PREVIOUS`, Redeploy, puis Réglages › Intégrations › **Rechiffrer avec la nouvelle clé** ; ensuite seulement, retire l'ancienne (détails : `docs/SECURITY.md` §12b) ;
    - jeton Instagram 60 jours : l'application crée une tâche 14 jours avant l'expiration ;
    - départ d'un prestataire ou changement d'ordinateur : refaire les jetons qu'il a pu voir.
- **Un seul propriétaire par ressource, et c'est Valemont :** comptes Google, portefeuille Meta, compte Vercel, Wise, Stripe. Activez la double authentification partout.
- **Sauvegardes :** la copie JSON téléchargeable contient des données personnelles, gardez-la dans un endroit privé. Une restauration se fait toujours dans un **nouveau** Sheet, jamais par-dessus la production.

Délais à anticiper (récapitulatif)

Quoi	Délai	En attendant
Vérification Google pour `gmail.send` (app Externe en production)	plusieurs jours à quelques semaines	App Interne (Workspace), ou mode Test avec vos deux adresses, ou brouillon Gmail pré-rempli
App Review Meta (commentaires/messages Instagram)	quelques jours à quelques semaines	Relais Make « Watch comments » vers Messages à traiter
API LinkedIn Community Management	long, non garanti	Exports .xls/.xlsx importés chaque semaine
Certificat du domaine Vercel	quelques minutes à quelques heures (DNS)	Adresse `*.vercel.app` pour tester (penser à ajouter les nouvelles URI Google ensuite)